出栈顺序

1. 元素a,b,c,d,e依次进入初始为空的栈，若元素进栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素d开头的序列个数是（ ）

A.3　B. 4　C. 5　D. 6

d先出栈，abc的顺序只能是逆序即cba，然后，e的位置可以在dcba中间的三个位置加上最后一个位置。即选B

栈的应用举例

- 数制转换
- 括号匹配
- 行编辑程序
- 迷宫求解
- 表达式求值

数制转换

十进制数转换为八进制数。采用对十进制数除8取余的方法，可得到八进制数的倒序。

$N = (N \text{ div } d) \times d + N \bmod d$

例如：　$(1348)_{10} = (2504)_8$，其运算过程如下：

计算顺序

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2

输出顺序

```
void conversion () {
    InitStack(S);
    scanf ("%d",N);
    while (N) {
      Push(S, N % 8);
      N = N/8;
    }
    while (!StackEmpty(S)) {
      Pop(S,e);
      printf ( "%d", e );
    }
} // conversion
```

栈的应用：括号匹配

- 括号：{，}；[，]；(，)

表达式	匹配?
(A+B)	Yes
{(A+B)×(C+D)}	Yes
{(x+y)×z	No
[2+3]+4}	No
{a+z]	No
5+6)÷(4	No
[()]	Yes
[(])	No

```
template <typename T> void
List<T>::merge ( ListNodePosi(T) & p, int n,
List<T>& L, ListNodePosi(T) q, int m )
{
ListNodePosi(T) pp = p->pred;
while ( 0 < m )
if ( ( 0 < n ) && ( p->data <= q->data ) )
{ if ( q == ( p = p->succ ) ) break; n--; }
else { insertB ( p, L.remove ( ( q = q->succ )->pred ) ); m--; }
p = pp->succ;}
```

代码分析器

策略：判断左括号和右括号个数是否相等？ ✗

栈的应用：括号匹配

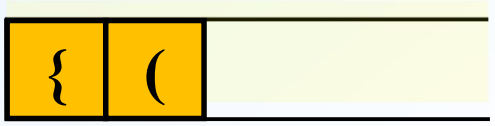
■ 括号：{，}；[，]；(，)

表达式	匹配?
(A + B)	Yes
{ (A + B) × (C + D) }	Yes
{ (x + y) × z	No
[2 + 3] + 4 }	No
{ a + z]	No
5 + 6) ÷ (4	No
[( )]	Yes
[( ])	No

■ 策略： 后开先闭！

- ✓ 从左往右扫描
- ✓ 碰到开（左）括号，入栈
- ✓ 碰到闭（右）括号，若栈顶不是对应的开括号，返回错误；若是，则出栈
- ✓ 最后必须栈空

(A + B) × (C + D)



栈的应用：括号匹配

代码实现

```
bool paren ( const char exp[], int lo, int hi ) {
                                        //表达式括号匹配检查，可兼顾三种括号
   Stack<char> S;                       //使用栈记录已发现但尚未匹配的左括号
   for ( int i = lo; i <= hi; i++ ) {          /* 逐一检查当前字符 */
      switch ( exp[i] ) {
                 //左括号直接进栈；右括号若与栈顶失配，则表达式必不匹配
         case '(': case '[': case '{': S.push ( exp[i] ); break;
         case ')': if ( ( S.empty() ) || ( '(' != S.pop() ) )
                                                  return false; break;
         case ']': if ( ( S.empty() ) || ( '[' != S.pop() ) )
                                                  return false; break;
         case '}': if ( ( S.empty() ) || ( '{' != S.pop() ) )
                                                  return false; break;
         default: break;                          //非括号字符一律忽略
   }
   return S.empty();
//整个表达式扫描过后，栈中若仍残留（左）括号，则不匹配；否则（栈空）匹配
}
```

行编辑程序

在用户输入一行的过程中，允许 用户输入出差错，并在发现有误时可以及时更正。

设立一个**输入缓冲区（栈）**，用以接受用户输入的一行字符，然后逐行存入用户数据区；并假设“#”为退格符，“@”为退行符。

假设从终端接受两行字符：

```
whli##ilr#e （s#*s)
outcha@putchar(*s=#++);
```

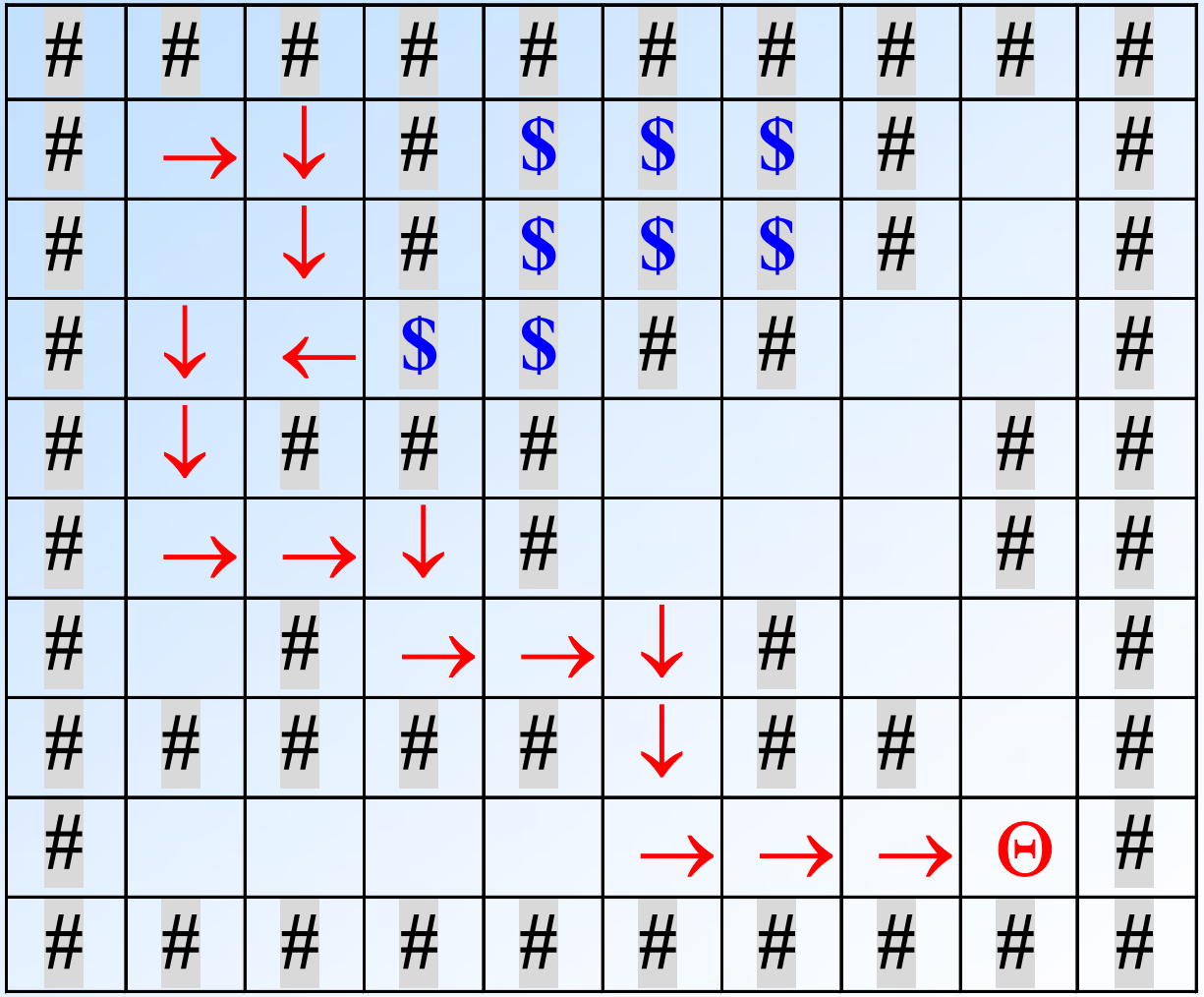
实际有效行为：

```
while (*s)
  putchar(*s++);
```

```
Void LineEdit(){
   InitStack(S);
   ch=getchar();
   while (ch != EOF) { //EOF为全文结束符
                       while (ch != EOF && ch != '\n') {
                           switch (ch) {
                                  case '#' : Pop(S, ch); break;
                                  case '@': ClearStack(S); break;
                                            // 重置S为空栈
                                   default : Push(S, ch);  break;
                                  }
                          ch = getchar();  // 从终端接收下一个字符
                            }//while
   将从栈底到栈顶的字符传送至调用过程的数据区;
   ClearStack(S);      // 重置S为空栈
   if (ch != EOF)  ch = getchar();
   } //while
   DestroyStack(S);
}
```

❖迷宫求解

通常用的是“穷举求解”的方法



❖迷宫路径算法的基本思想

若当前位置“可通”，则纳入路径，继续前进;

若当前位置“不可通”，则后退，换方向继续探索;

若四周“均无通路”，则将当前位置从路径中删除出去。

设定当前位置的初值为入口位置；

do {

若当前位置可通，

则 {将当前位置插入栈顶；

若该位置是出口位置，则算法结束；

否则切换当前位置的东邻方块为新的当前位置；

}

……

否则{

若栈不空且栈顶位置尚有其他方向未被探索，

则设定新的当前位置为：沿顺时针方向旋转

找到的栈顶位置的下一相邻块；

若栈不空但栈顶位置的四周均不可通，

则{

删去栈顶位置；// 从路径中删去该通道块

若栈不空，则重新测试新的栈顶位置，

直至找到一个可通的相邻块或出栈至栈空；

}

}

} while (栈不空）；

```
typedef struct {
   int  ord;                    //通道块在路径上的序号
   PosType seat;                //通道块在迷宫的坐标
   int  di;                     //走向下一通道块的方向
}SElemType;                     //栈元素

Status MazePath (MazeType maze, PosType
   start, PosType end) {
   InitStack(S);
   curpos=start;        //当前位置为入口位置
   curstep=1;           //探索第一步
```

```
do {
    if (Pass(curpos)) {  //可通过且未走过
            FootPrint(curpos);  //记已走过标记
            e=(curstep, curpos, 1);
            Push (S, e);
            if (curpos == end) return (TRUE);
            curpos = NextPos ( curpos, 1); //东邻
            curstep++; //探索下一步
    }//if
```

如果当前结点可通过且未走过，
入栈并生成下一个结点

```
typedef struct {
   int ord;              //序号
   PosType seat;         //坐标
   int di;               //方向
}SElemType;              //栈元素
```

```
        else {
            if (!StackEmpty(S)) {
                Pop(S, e);      找到一个还有未尝试通路的回溯点
                while (e.di==4 && !StackEmpty(S)) {
                        MarkPrint(e.seat); Pop(S,e);
                        //记不能通过标记，并退回
                }//while
                if(e.di<4) {
                        e.di++; Push(S, e);
                        curpos = NextPos(e.seat, e.di);
                        //设定当前位置为该方向上的相邻块
                }//if
            }//if
        }//else
    }while (!StackEmpty(S)); //如果栈空没有可行路线
    return (FALSE);
}//MazePath
```

否则当前结点不可通过或者已走过，需要退栈回溯

如果找不到未尝试通路的回溯点，则无解

表达式求值

限于二元运算符的表达式定义:

表达式 ::= (操作数) + (运算符) + (操作数)

操作数 ::= 简单变量 | 表达式

简单变量 :: = 标识符 | 无符号整数

Exp = S1 + OP + S2

前缀表示法OP + S1 + S2

中缀表示法 S1 + OP + S2

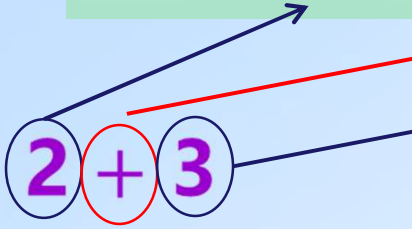
后缀表示法 S1 + S2 + OP

例如:	Exp = a × b + (c – d / e) × f
前缀式:	+ × a b × – c / d e f
中缀式:	a × b + (c – d / e) × f
后缀式:	a b × c d e / – f × +

栈的应用：表达式求值

■ 常规（中缀）表达式

＜操作数＞＜操作符＞＜操作数＞



A − b

P × 2

(A+2) × (2-B)

2 ^ 3 ^ 2

■ 优先级：

1) 括号：　(), [ ], { }
2) 指数运算符　(从右往左)
3) 乘除运算符　(从左往右)
4) 加减运算符　(从左往右)

操作符：＋ − × ÷ ^

中缀表达式缺陷：计算机难以处理，需借助括号强制改变优先级

栈的应用：表达式求值

■ 前缀（Polish Notation）表达式 [1924']

<操作符><操作数><操作数>

中 缀	前 缀
2 + 5	+ 2 5
a × b	× a b
(x + y) × z	× + x y z
a + b × c	+ a × b c

中缀表达式中，操作数有二义性（可前可后结），需使用括号及优先级定义确定

前缀表达式中，操作数对应的操作符确定唯一,不需使用优先级及括号

■ 人工转换

(x + y) × z

(+ x y) × z

× (+ x y) z

× + x y z

a + (b × c)

a + (× b c)

+ a (× b c)

+ a × b c

栈的应用：表达式求值

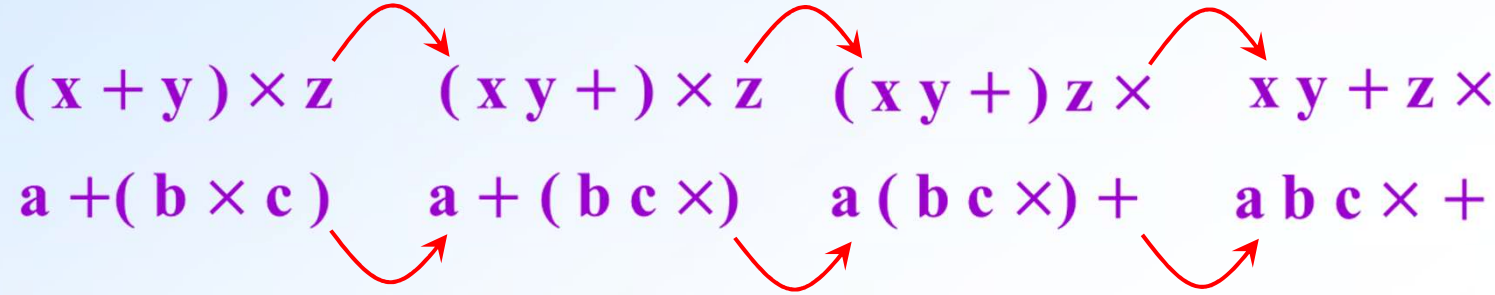
- 后缀（Reverse Polish Notation）表达式 1950'

＜操作数＞＜操作数＞＜操作符＞

中 缀	前 缀	后 缀
2 + 5	+ 2 5	2 5 +
a × b	× a b	a b ×
(x + y) × z	× + x y z	x y + z ×
a + b × c	+ a × b c	a b c × +

适合人类阅读　　适用于计算机

- 手动转换



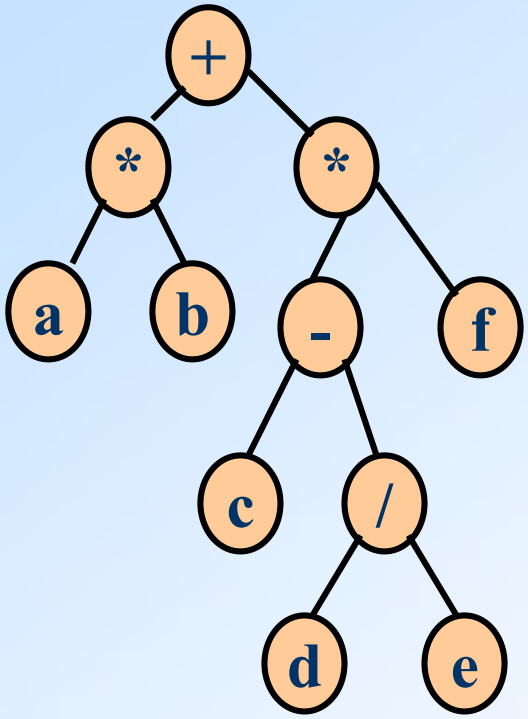
表达式标识方法

例如：　Exp = a × b + (c – d / e) × f

前缀式：　+ × a b × – c / d e f

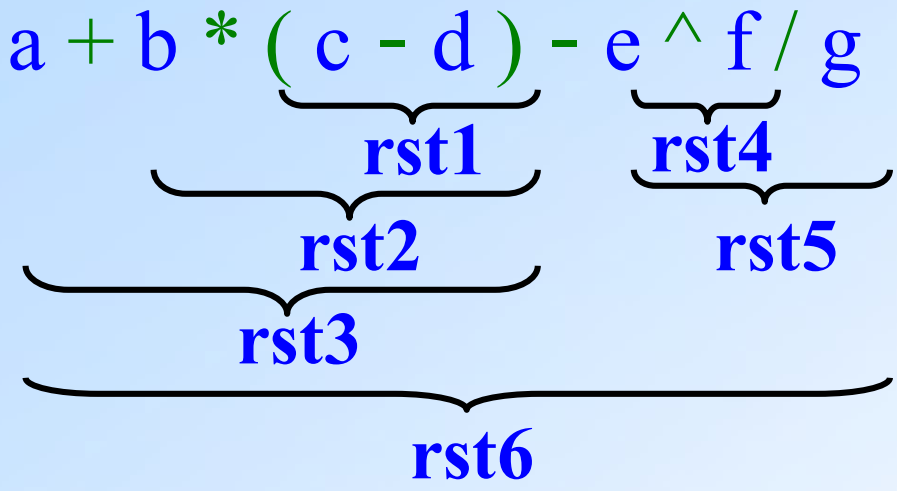
中缀式：　a × b + (c – d / e) × f

后缀式：　a b × c d e / – f × +



后缀表达式，又称逆波兰式，指不包含括号，运算符放在两个运算对象的后面，所有的计算按运算符出现的顺序，严格从左向右进行（不再考虑运算符的优先规则）

表达式的中缀表示



- 表达式中相邻两个操作符的计算次序为：
  - 优先级高的先计算；
  - 优先级相同的自左向右计算；
  - 当使用括号时从最内层括号开始计算。

- 一般表达式的操作符有4种类型：

1 算术操作符　如双目操作符（+、－、*、/和%）以及单目操作符（－）

2 关系操作符　包括<、<=、==、!=、>=、>。这些操作符主要用于比较

3 逻辑操作符　如与(&&)、或(||)、非(!)

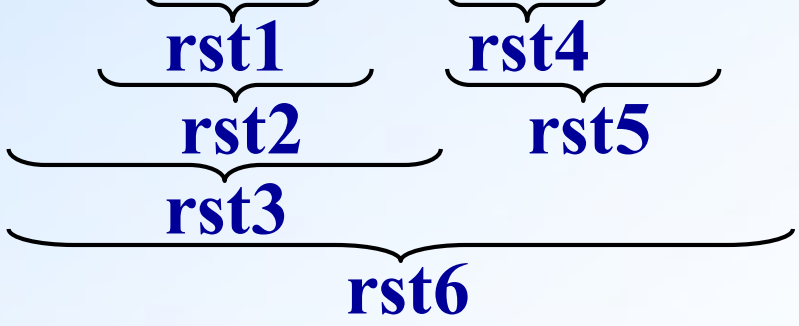
4 括号‘(’和‘)’　它们的作用是改变运算顺序

C++中操作符的优先级

优先级	操作符
1	单目-、!
2	*、/、%
3	+、-
4	<、<=、>、>=
5	==、!=
6	&&
7	||

后缀表示法计算表达式的值

- 从左向右顺序地扫描表达式，并用一个栈暂存扫描到的操作数或计算结果。
- 在后缀表达式的计算顺序中已隐含了加括号的优先次序，括号在后缀表达式中不出现。
- 计算例　a b c d - * + e f ^ g / -



通过后缀表示计算表达式值的过程

- 顺序扫描表达式的每一项，根据它的类型做如下相应操作：
  - 若该项是操作数，则将其压栈；
  - 若该项是操作符<op>，则连续从栈中退出两个操作数Y和X，形成运算指令X<op>Y，并将计算结果重新压栈。
- 当表达式的所有项都扫描并处理完后，栈顶存放的就是最后的计算结果。

步	输入	类 型	动 作	栈内容
1			置空栈	空
2	A	操作数	进栈	A
3	B	操作数	进栈	AB
4	C	操作数	进栈	ABC
5	D	操作数	进栈	ABCD
6	−	操作符	D、C 退栈，计算 C−D，结果 r1 进栈	ABr1
7	*	操作符	r1、B 退栈，计算 B*r1，结果 r2 进栈	Ar2
8	+	操作符	r2、A 退栈，计算 A+r2，结果 r3 进栈	r3

步	输入	类 型	动 作	栈内容
9	E	操作数	进栈	r3E
10	F	操作数	进栈	r3EF
11	^	操作符	F、E 退栈，计算 E^F，结果 r4 进栈	r3r4
12	G	操作数	进栈	r3r4G
13	/	操作符	G、r4 退栈，计算 r4/E，结果 r5 进栈	r3r5
14	+	操作符	r5、r3 退栈，计算 r3+r5，结果 r6 进栈	r6

```
void Calculator :: Run ( ) {
   char ch;   double newoperand;
   while ( cin >> ch,  ch != '=' ) {
      switch ( ch ) {
        case '+' :  case '–' :   case '*' :   case '/' :
            DoOperator ( ch );  break;   //计算
        default :  cin.putback ( ch );
                          //将字符放回输入流
                   cin >> newoperand;  //读操作数
                   AddOperand ( newoperand );
                   //操作数进栈
      }
   }
}
```

注意：放回输入流中重新读取为double型数值再入栈

利用栈将中缀表示转换为后缀表示

- 使用栈可将表达式的中缀表示转换成它的前缀表示和后缀表示。
- 为了实现这种转换，需要考虑各操作符的优先级。

算符间优先级

优先级在栈内栈外是不一样的。+-*/ 在栈内的优先级要高

c1 栈内 \ c2 栈外	+	-	*	/	(	)	#
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=

各个算术操作符的优先级

操作符 ch	#	(	^	*, /, %	+, –	)
isp (栈内)	0	1	7	5	3	8
icp (栈外)	0	8	6	4	2	1

- isp叫做栈内(in stack priority)优先数
- icp叫做栈外(in coming priority)优先数
- 操作符优先数相等的情况出现在括号配对或栈底的#号与输入流最后的#号配对
- ）与（，（与#，#与）不能相继出现

保持级差，同级进栈升高，（ 入栈降至最低

从中缀式求得后缀式方法

1) 设立暂存运算符的栈;
2) 设表达式的结束符为“#”, 予设运算符栈的栈底为“#”
3) 若当前字符是操作数，则直接发送给后缀式;(后缀与前缀式操作数的顺序是相同的)
4) 若当前运算符的优先数高于栈顶运算符，则进栈;
5) 否则，退出栈顶运算符发送给后缀式;
6) “(” 对它之前后的运算符起隔离作用，
“)”为自左括弧开始的表达式的结束符。

```
void postfix ( expression e ) {
  stack<char> s;   char ch, op;
  s.Push ( '#' );              cin.Get ( ch );
 while ( ! s.IsEmpty( ) && ch != '#' )
      if ( isdigit ( ch ) )     //操作数，直接输出
        { cout << ch;  cin.Get ( ch ); }
      else {
        if ( isp ( s.GetTop( ) ) < icp ( ch ) )  //栈外优先级高则入栈
          { s.Push ( ch );  cin.Get ( ch ); }
        else if ( isp ( s.GetTop( ) ) > icp ( ch ) )  //栈内优先级高则出栈
          { op = s.GetTop ( );  s.Pop( );   //注意，这里并没有继续读
            cout << op;}
         else { op = s.GetTop ( );  s.Pop( ); //栈内外优先级相等
                if ( op == '(' ) cin.Get ( ch ); }  //括号配对
      }
}
```

操作符 ch	#	(	^	*, /, %	+, −	)
isp (栈内)	0	1	7	5	3	8
icp (栈外)	0	8	6	4	2	1

中缀式: $a \times b + (c - d / e) \times f$

后缀式: $a\ b \times c\ d\ e / - f \times +$

中缀算术表达式求值

为实现算符优先算法，在这里用了两个工作栈。一个存放算符OPTR，另一个存放数据OPND。算法思想是：

（1）首先置数据栈为空栈，表达式起始符“#”为算符栈的栈底元素

（2）自左向右扫描表达式，读到操作数进OPND栈，读到运算符，则和OPTR栈顶元素比较（栈顶元素为c1，读到的算符为c2);

若c1<c2,则c2进栈继续扫描后面表达式；

若c1 = c2,则（“=”），即括号内运算结束，将c1出栈，并且c2放弃,继并在操作数栈扫描后面表达式；

若c1>c2,则将c1出栈，并在操作数栈取出两个元素a和b按c1做运算，运算结果进OPND.

重复直到表达式求值完毕。

为使两个算符比较方便,给算符设置优先级,如下表,其中c1为栈内元素,c2为栈外元素

算符	栈内优先级	栈外优先级
* /	4	3
+ -	2	1
(	0	5
)	5	0
#	-1	-1

例如：表达式3*(7-2),求值过程如下表：

步骤	OPTR栈	OPND栈	输入字符	主要操作
1	#		3*(7-2)#	PUSH(OPND,'3')
2	#	3	*(7-2)#	PUSH(OPTR,'*')
3	# *	3	(7-2)#	PUSH(OPTR,'(')
4	# * (	3	7-2)#	PUSH(OPND,'7')
5	# * (	3 7	-2)#	PUSH(OPTR,'-')
6	# * (-	3 7	2)#	PUSH(OPND,'2')
7	# * (	3 7 2	)#	operate('7','-','2')
8	# * (	3 5	)#	POP(OPTR)消去括号
9	# *	15	#	operate('3','*','5')
10	#	15	#	结束

算符比较算法

```
char Precede(char c1,char c2)
{int c_temp1,c_temp2;
  switch(c1)
{  case  '*':
 case  '/':c_temp1=4;break;
case  '+':
 case  '-':c_temp1=2;break;
 case  '(':c_temp1=0;break;
 case  ')':c_temp1=5;break;
 case  '#':c_temp1=-1;}
```

```
switch(c2)
{  case  '*':
   case  '/':c_temp2=3;break;
   case  '+':
   case  '-':c_temp2=1;break;
   case  '(':c_temp2=5;break;
   case  ')':c_temp2=0;break;
   case  '#':c_temp2=-1;}
if(c_temp1<c_temp2)   return('<');
if(c_temp1=c_temp2)   return('=');
if(c_temp1>c_temp2)   return('>');
}
```

```
switch(c1)
{  case  '*':
 case  '/':c_temp1=4;break;
case  '+':
 case  '-':c_temp1=2;break;
 case  '(':c_temp1=0;break;
 case  ')':c_temp1=5;break;
 case  '#':c_temp1=-1;}
```

算符优先算法求中缀表达式值

```
OperandType EvaluateExpression() {
    InitStack (OPTR); Push ( OPTR,'#');
    InitStack (OPND); c=getchar();
    while (c!='#'|| GetTop(OPTR)!='#') {
        if(!In(c,OP)){Push(OPND,c);  //操作数
                c=getchar();}
        else
                switch(Precede(GetTop(OPTR),c){
                case '<'://栈顶元素优先权低
                        Push(OPTR,c); c=getchar();
                        break;
```

```
                case '=':脱括号
                    Pop(OPTR,x); c=getchar();
                    break;
                case '>':退栈并计算
                    Pop(OPTR,theta);
                    Pop(OPND,b); Pop(OPND,a);
                    Push(OPND, Operate(a,theta,b));
                    break; //注意没有c=getchar();
            }//switch
    }//while
    return GetTop(OPND);
}//EvaluateExpression
```

中缀表达式转后缀表达式(简化)

- 按照运算的先后顺序将表达式全部添加上括号

  (a+b)*c*(d-e/f)----> (((a+b)*c)*((d-(e/f))))

- 后缀表达式，从里到外将所有运算符都拿到右括号的右边(前缀表达式，把运算符拿到括号左边)

  (((ab)+c)*((d(ef)/)-))*

- 将所有括号都去掉
  - ab+c*def/-*
- ----------
- 作者：Casionx
- 来源：CSDN
- 原文：https://blog.csdn.net/summerxiachen/article/details/77073320
- 版权声明：本文为博主原创文章，转载请附上博文链接！

练习

- 写出下列中缀表达式的后缀形式：
- (1) A + B + C + D

A. ABCD+++　　B. AB+C+D+　　C. AB+CD++

- (2) (A + B) * D + E / (F + A * D) + C

练习

- 写出下列中缀表达式的后缀形式：
- (1) A + B + C + D

A. ABCD+++　　B. AB+C+D+　　C. AB+CD++

- (2) (A + B) * D + E / (F + A * D) + C

A B + D * E F A D * + / + C +

- **课堂练习：如何实现浏览器的前进与后退功能**

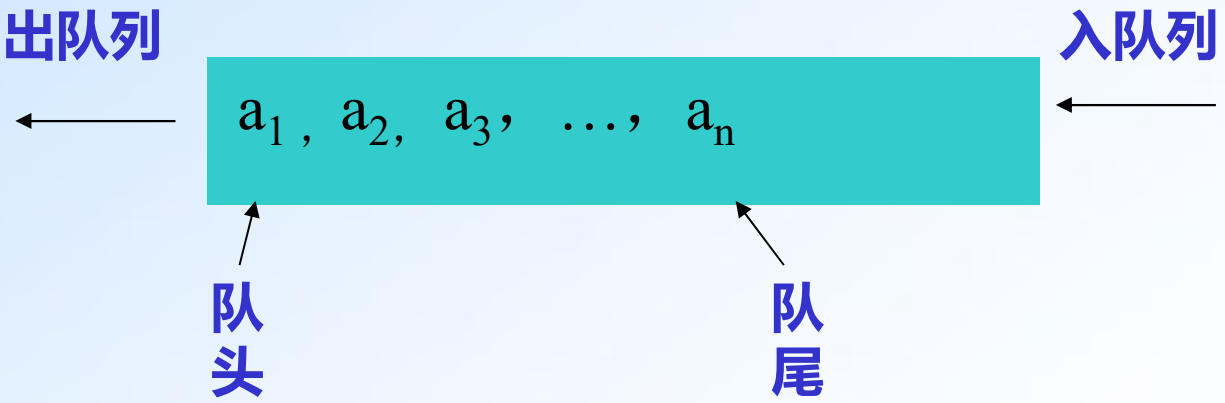
用两个栈X 和 Y，把首次浏览的页面依次压入栈 X，当点击后退按钮时，再依次从栈 X 中出栈，并将出栈的数据依次放入栈 Y。当我们点击前进按钮时，我们依次从栈 Y 中取出数据，放入栈 X 中。当栈 X 中没有数据时，那就说明没有页面可以继续后退浏览了。当栈 Y 中没有数据，那就说明没有页面可以点击前进按钮浏览了。

队列

定义:只允许在表的一端进行插入，而在另一端删除元素的线性表。

在队列中，允许插入的一端叫队尾 (rear)，允许删除的一端称为队头(front)。

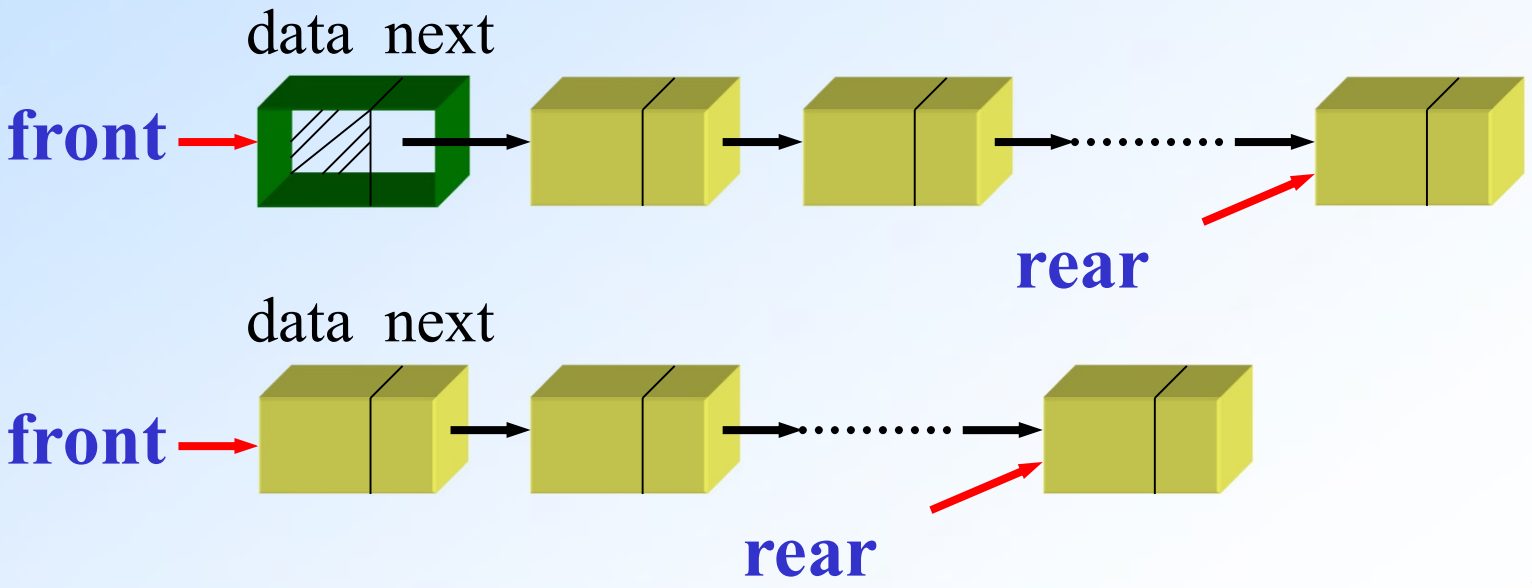
特点： 先进先出 (FIFO)

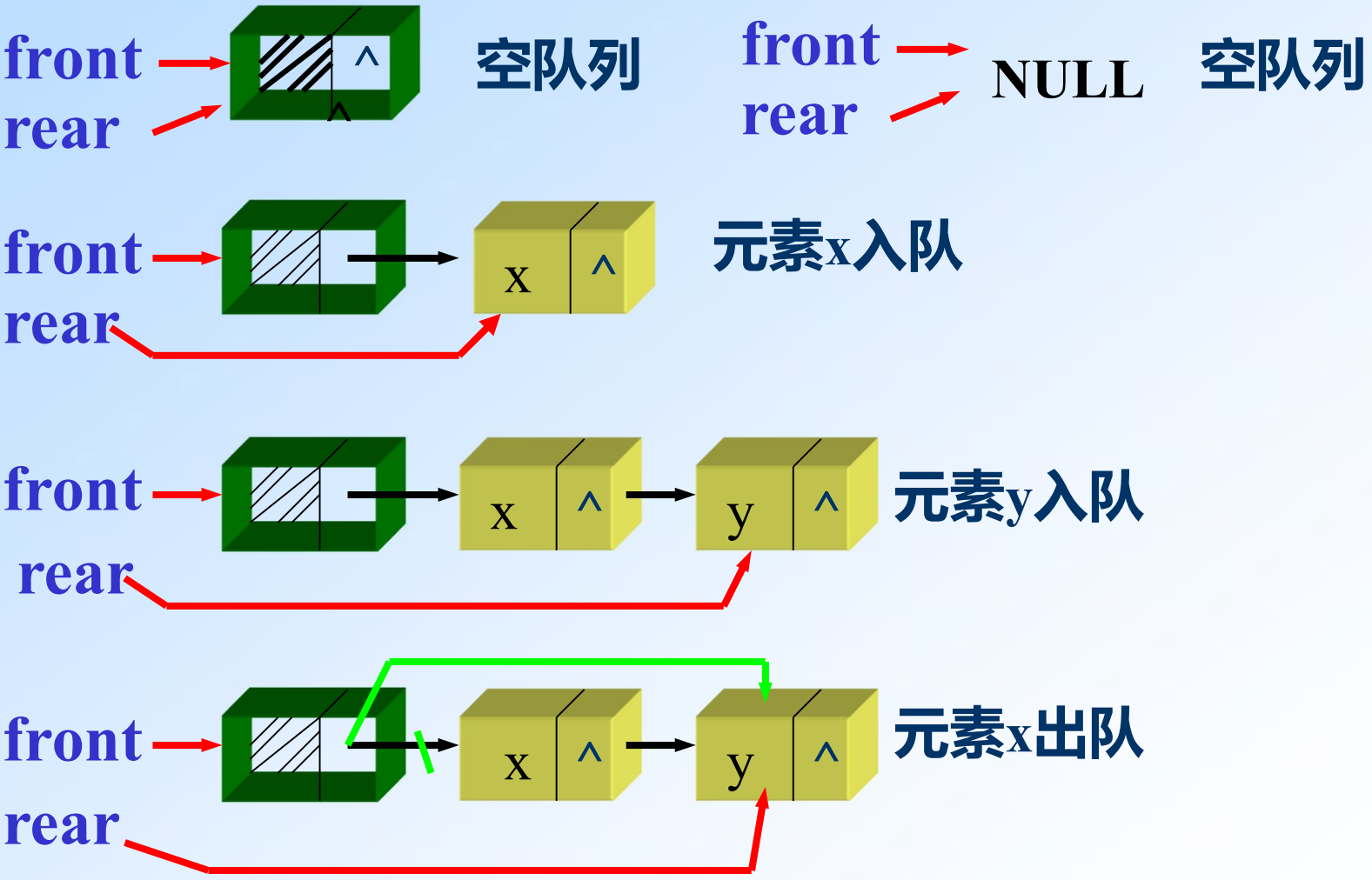


链队列：队列的链式表示

链队列中，有两个分别指示队头和队尾的指针。

链式队列在进队时无队满问题，但有队空问题。




front
rear
^
^
空队列
front
rear
NULL
空队列
front
rear
x
^
元素x入队
front
rear
x
^
y
^
元素y入队
front
rear
x
^
y
^
元素x出队

链式队列的表示及基本操作(C语言)

```
typedef struct QNode{ //链队列结点的类型定义
    QElemType  data;  //用于存放队列元素
    struct QNode *next;  //用于存放元素直接后继
                         //结点的地址
}QNode,*QueuePtr;

typedef struct
{  //链队列的表头结点的的类型定义
        QueuePtr front;  //队头指针，指向链表的头结点
        QueuePtr rear;   //队尾指针，指向队尾结点
}LinkQueue;
```

链式队列的表示及基本操作(C语言)

```
Status InitQueue_L(LinkQueue &Q) {//建一个空队列
Q.front=Q.rear=(QueuePtr)malloc(sizeof(QNode));
//为链队列的头结点分配空间
    if (!Q.front) exit(OVERFLOW);
    Q.front->next=NULL;
     return OK;
} //InitQueue_L
```

链式队列的表示及基本操作(C语言)

```
Status DestroyQueue_L(LinkQueue &Q)  {
        //销毁队列Q
       if (!Q.front) return ERROR; // 链队列不存在
        // 回收链队列的所有元素结点空间
       while(Q.front->next) {
           p=Q.front->next;
          Q.front->next=p->next;
          free(p);
       }
       free(Q.front );            // 回收链队列头结点空间
      Q.front=Q.rear=null;
       return OK;
}//DestroyQueue_L
```

如果是清空的话要保留头结点，指针都指向头结点

销毁的时候需要每个结点释放（包括头结点）

链式队列的表示及基本操作(C语言)

```
//在链队列Q队尾，插入新的队尾结点
Status EnQueue_L(LinkQueue &Q, QElemType &e ) {
    //为新元素e分配结点空间
    p=(QueuePtr)malloc(sizeof(QNode));
    if (!p) exit (OVERFLOW);     //存储分配失败
    p->data=e; p->next=NULL;
    Q.rear->next=p;    //插入新队尾结点
    Q.rear=p;   //修改队尾指针
     return  OK;
 }
```

链式队列的表示及基本操作(C语言)

```
//若队列不空，则删除Q的队头元素结点，用e返回其值，并返回OK；否则返回ERROR
Status DeQueue_L(LinkQueue &Q, QElemType &e)  {
      if (Q.front = =Q.rear) return ERROR;
          //若队列空，返回ERROR
      p=Q.front->next;              // p指向队头元素结点
      e=p->data;
      Q.front->next=p->next; // 修改链队列头结点指针
 // 对于链队列只有一个元素结点的情况,要同时修改队尾指针
      if(Q.rear= =p)  Q.rear=Q.front;  //避免p被释放后，rear
                                       成为悬垂指针
      free(p);
      return OK;
}
```

顺序队列

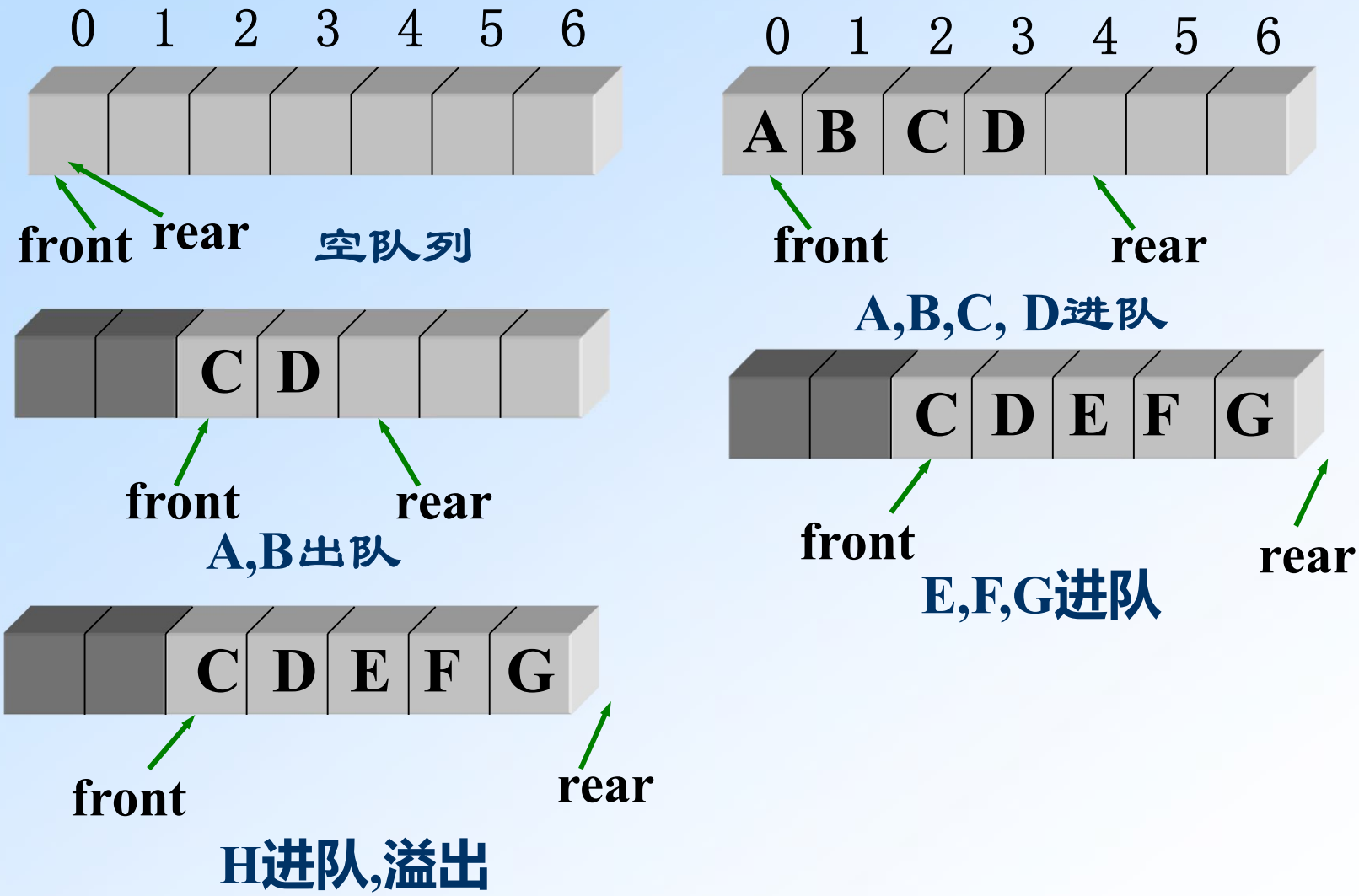
队列的顺序存储表示。用一组地址连续的存储单元依次存放从队列头到队列尾的元素，指针front和rear分别指示队头元素和队尾元素的位置。

- 插入新的队尾元素，尾指针增1， rear = rear + 1
- 删除队头元素，头指针增1， front = front + 1
- 在非空队列中，头指针始终指向队列头元素，而尾指针始终指向队列尾元素的下一个位置
- 队满时再进队将溢出，队满条件：

  rear==maxsize

队头指针与栈底指针不同，队头指针要指向可用的数组位置

队列的进队和出队


0 1 2 3 4 5 6
front
rear
空队列
A B C D
front
rear
A,B,C, D进队
C D
front
rear
A,B出队
C D E F G
front
rear
E,F,G进队
C D E F G
front
rear
H进队,溢出

循环队列 (Circular Queue)

将顺序队列臆造为一个环状的空间，形成循环(环形)队列

队头、队尾指针加1，可用取模(余数)运算实现。

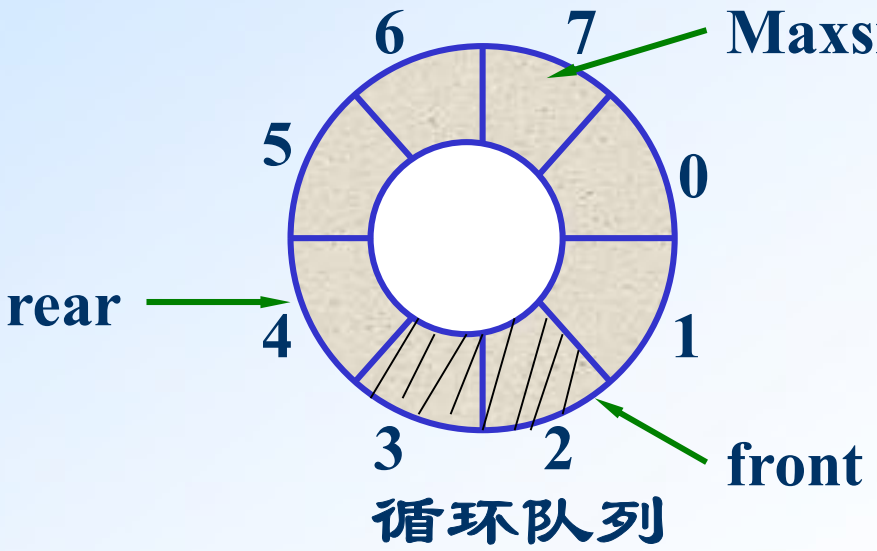
队头指针进1：front = (front+1) %maxsize;

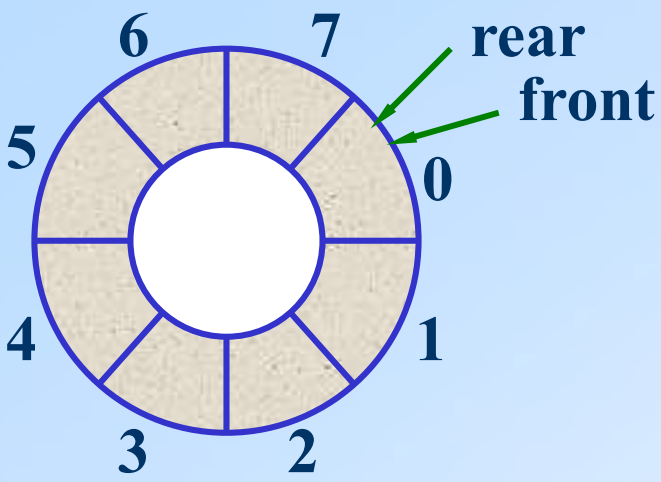
队尾指针进1：rear = (rear+1) % maxsize;

队列初始化：front = rear = 0;

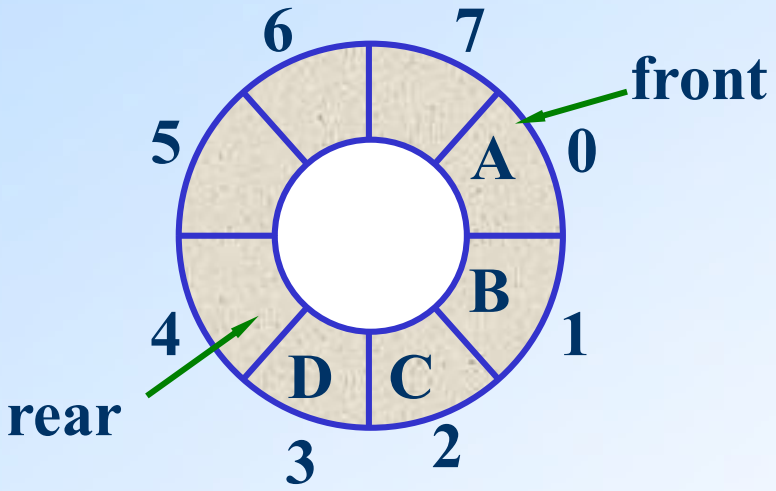
队空条件：front == rear;

队满条件：(rear+1) % maxsize == front; （见下页解释）

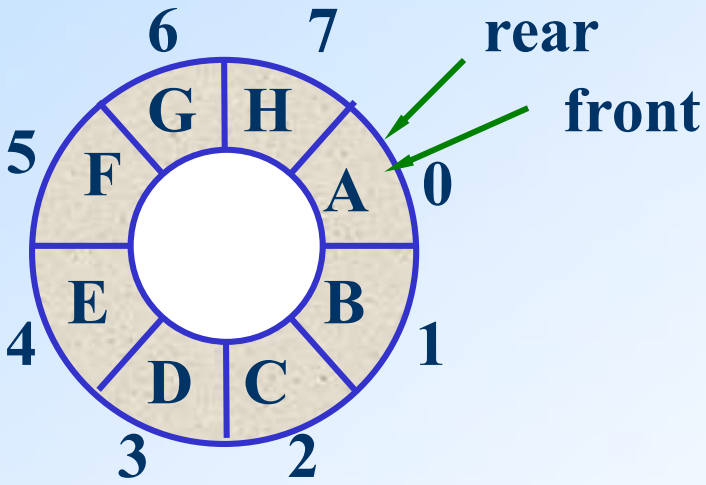




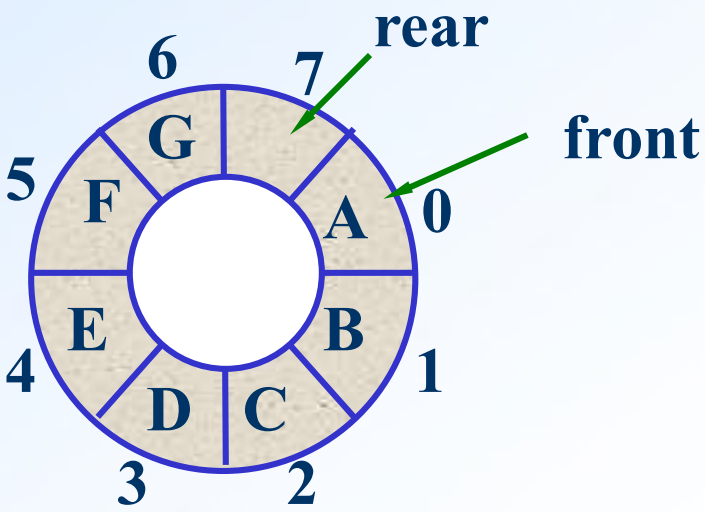
空队列



一般情况



队满(错误)



队满(正确)

Rear指向第一个空位置

循环队列的表示及基本操作(C语言)

```
#define  MAXSIZE 100  //  最大队列长度
typedef struct{
   QElemType  *base;  //初始化时动态分配存储空间的基址
   int front;    //队头指针，指向队头元素
   int rear;     //队尾指针，指向队尾元素的下一个位置
}SqQueue;

Status InitQueue_Sq(SqQueue &Q) {//构造一个空队列Q
      Q.base=(ElemType * )malloc (MAXSIZE*sizeof (ElemType));
      if (!Q.base) exit (OVERFLOW);    //存储分配失败
      Q.front = Q.rear=0;
      Return OK;
}  // InitQueue_Sq
```

循环队列的表示及基本操作(C语言)

```
//将元素e插入队尾
Status EnQueue_Sq(SqQueue  &Q, QElemType  e )
{
      if ((Q.rear+1)%MAXSIZE= =Q.front)
            return ERROR ;
      Q.base[Q.rear] = e ;      // 将元素e插入队尾
      Q.rear= (Q.rear+1)%MAXSIZE;   // 修改队尾指针
      return OK;
} // EnQueue_Sq
```

循环队列的表示及基本操作(C语言)

```
Status DeQueue_Sq(SqQueue  &Q, QElemType &e ){
    //删除队头元素，用e返回其值,并返回OK；否
    //则返回ERROR
   if ((Q.rear= =Q.front) return ERROR ;
   e =Q.base[Q.front] ;
   Q.front= (Q.front+1)%MAXSIZE; // 修改队头指针
        return OK;
  }  // EnQueue_Sq
```

优先级队列 (Priority Queue)

优先级队列 是不同于先进先出队列的另一种队列。每次从队列中取出的是具有最高优先权的元素

例如下表：任务的优先权及执行顺序的关系

通常采用堆数据结构来实现

任务编号	1	2	3	4	5
优先权	20	0	40	30	10
执行顺序	3	1	5	4	2

数字越小，优先权越高，设不存在同优先级元素